Documentation de l'extension TRIGO

Groupe 36

Janvier 2026

Contents

1 Introduction-Calculs numériques par ordinateur **3**

2 Approximation polynomiale pour des fonctions régulières **4**
- 2.1 Formule de Taylor-Young 4
- 2.2 Application aux fonctions trigonométriques 4

3 Première méthode : Relations de trigonométrie **5**
- 3.1 Explications 5
- 3.2 Tests 5

4 Affinement avec méthode de Horner **6**
- 4.1 Explication de la méthode 6
- 4.2 Application aux fonctions trigonométriques 7
- 4.3 Réduction de l'angle 7
- 4.4 Tests 7

5 Implémentation finale de cos et sin **8**
- 5.1 Réduction d'argument 8
- 5.2 Cas des nombres très proches de 0 12
- 5.3 Cas des très grands nombres 12

6 Gestion des fonctions arctangente et arcsinus **13**
- 6.1 Implémentation de la fonction arctangente 13
- 6.2 Implémentation de la fonction arcsinus 14
- 6.3 Implémentation de la fonction racine carrée 15
- 6.4 Tests des fonctions arctan et arcsin 16

7 Algorithme de calcul de l'ulp **18**
- 7.1 Explication de l'algorithme 18
- 7.2 Ecriture de l'algorithme en Deca et tests 19

8 Tests de la librairie en Deca et conclusion **20**
- 8.1 Tests de l'implémentation en Deca de la librairie avec notre compilateur 20
- 8.2 Conclusion sur la librairie 22

9 Bibliographie **22**

1 Introduction-Calculs numériques par ordinateur

Un ordinateur dispose d'un nombre fini de bits pour représenter des nombres réels infinis, il doit alors toujours arrondir les résultats.

Il y a deux façons de mesurer l'erreur :

- Ulps (Units in the Last Place) : C'est l'écart entre deux nombres consécutifs que l'ordinateur peut représenter. C'est la plus petite unité de mesure pour cet ordinateur.

- Erreur relative (epsilon) : C'est la différence entre le nombre réel et son approximation, divisée par le nombre réel.

Il y a donc un danger d'annulation, si deux nombre très proches déjà arrondis sont soustraits alors le résultat ne sera pas du tout précis à cause des erreurs accumulées d'arrondis.

Il y a une norme (IEEE 754) qui assure la portabilité des programmes pour écrire les nombres flottants.

Dans notre contexte, un flottant est écrit avec 32 bits. Un bit est réservé pour le signe (0 pour un nombre positif et 1 pour un négatif). L'exposant est écrit avec 8 bits, l'exposant "0" sera écrit 127 (biais), et donc la plage réelle de l'exposant va de -126 à +127, (en réservant le 0 et le 255 binaire par la norme IEEE 754). Les derniers 23 bits sont réservés pour la mantisse, ie les chiffres signicatifs du nombre.

La formule mathématique d'un nombre flottant est calculée ainsi :

$$V = (-1)^s \times (1, m) \times 2^{E-127}$$

avec :
- s le bit de signe.
- m la mantisse à laquelle on ajoute un "1" caché pour gagner de la place (en notation scientifique un nombre commence de 1 à 9).
- E l'exposant réelle.
- 127 représente le biais.

2 Approximation polynomiale pour des fonctions régulières

2.1 Formule de Taylor-Young

Soit I intervalle de $\mathbb{R}$.

Soient $a \in I$ et $f : I \to \mathbb{R} \in C^{n-1}$ au voisinage de a tq $f^{(n)}(a)$ existe. Alors la formule de Taylor-Young permettant d'approximer f par ses dérivées successives s'écrit:

$$f(x) =_a f(a)+f'(a)(x-a)+f''(a)\frac{(x-a)^2}{2}+\cdots+f^{(n)}(a)\frac{(x-a)^n}{n!}+o((x-a)^n)$$

avec $o((x-a)^n) = (x-a)^n\epsilon(x)$ tq $\epsilon(x) \underset{x\mapsto a}{\to} 0$.

2.2 Application aux fonctions trigonométriques

Les fonctions sinus, cosinus et arctangente peuvent être approximées en 0 à n'importe quel ordre puisqu'elles sont C^∞ sur $\mathbb{R}$.

Leurs dérivées sont faciles à calculer et à évaluer en 0, on peut remarquer aussi que par parité du cosinus (respectivement imparité du sinus et d'arctangente) seulement les coefficients d'ordre pair seront non nuls (respectivement d'ordre impair). Ainsi à l'ordre 10 par exemple ça nous donne :

$$cos(x) =_0 1 - \frac{x^2}{2} + \frac{x^4}{4!} - \frac{x^6}{6!} + \frac{x^8}{8!} - \frac{x^{10}}{10!} + o(x^{10})$$

$$sin(x) =_0 x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \frac{x^9}{9!} + o(x^{10})$$

$$\arctan(x) =_0 x - \frac{x^3}{3} + \frac{x^5}{5} - \frac{x^7}{7} + \frac{x^9}{9} + o(x^{10})$$

Et quand x va être proche de 0, on va pouvoir approximer les valeurs des fonctions par ces polynômes.

3 Première méthode : Relations de trigonométrie

3.1 Explications

Au voisinage de 0, cos et sin peuvent être approximés par leur polynôme de Taylor. Or, ceci est valable pour un intervalle proche de 0. Pour des valeurs plus éloignées, on doit se ramener au voisinage de 0. Pour faire ça on peut utiliser les formules de trigonométrie.

Pour calculer cosinus, on utilise : $cos(2x) = 2cos^2(x) - 1.$, puis on itère jusqu'à ce que x soit assez proche de 1 (j'ai pris une précision de 10^{-3}).

En approximant avec son polynôme de Taylor à l'ordre 8 on a des résultats peu précis pour des petites valeurs de x et des résultats complètement abberants pour des grandes valeurs de x.

Le problème est que, comme expliqué dans la partie 1, l'ordinateur approxime chaque valeur (ici sur 32 bits), et ici on fait un nombre énorme d'opérations pour des grands nombres (pour se ramener dans l'intervalle voulu). Ainsi l'erreur se propage et le résultat final sera complètement faux.

3.2 Tests

Table 1: Performances de la méthode naïve pour le calcul du cosinus sur différents intervalles.

Intervalle	**Métrique**	**Naive**
$[-100, 100]$	Err. Abs. Max	1.99
	Err. ULP Moy.	6.85×10^7
	Err. ULP Max	2.08×10^{12}
	Taux d'échec	99.998 %
$[-10\,000, 10\,000]$	Err. Abs. Max	2.00
	Err. ULP Moy.	6.43×10^7
	Err. ULP Max	1.54×10^{12}
	Taux d'échec	100.000 %
$[-100\,000, 100\,000]$	Err. Abs. Max	2.00
	Err. ULP Moy.	6.31×10^7

Suite sur la page suivante...

Table 1: Performances de la méthode naïve... (suite)

Intervalle	**Métrique**	**Naive**
	Err. ULP Max	1.01×10^{12}
	Taux d'échec	100.000 %

Ainsi, on observe qu'avec cette méthode, le critère de précision (précision inférieure à 1 ULP) est très rarement voire pas du tout respecté. L'aspect naïf de cet algorithme réside dans le fais que lorsqu'on calcule les différentes puissances du développement de Taylor, le calcul est naïf, pour résoudre ce problème, on peut utiliser la méthode de Horner qui permet d'effectuer ces calculs en limitant les erreurs d'arrondies, de plus la méthode permet d'utiliser l'instruction FMA qui va également limiter ces erreurs. Les réultats incorrects sont aussi dû aux divisions successives avec arrondi à chaque fois qui vont propager l'erreur.

Remarque : Les tests ont été réalisés en Java.

4 Affinement avec méthode de Horner

4.1 Explication de la méthode

La méthode de Horner est une technique mathématique permettant de faire moins de calculs pour trouver la valeur en un point d'un polynôme.

En fait, il n'est pas optimal de calculer tous les termes de chaque ordre un par un d'un polynôme puis de les additionner. Par exemple, pour un polynôme $P(x) = a_n x^n + \cdots + a_0$, pour calculer $a_n x^n$, on peut se servir de la valeur de $a_{n-1}x^{n-1}$ déjà calculée et donc de x^{n-1}.

Pour ce même polynôme l'algorithme de Horner permet d'écrire, pour un point donné x:

$$P(x) = (\cdots(((a_n x^n + a_{n-1})x + a_{n-2})x + a_{n-3}) \cdot \cdot)x + a_0$$

Ce calcul nécessitera simplement n multiplications et n additions, ce qui est beaucoup plus simple/rapide que n additions et $\frac{n(n+1)}{2} + n$ multiplications si on avait laissé le polynôme dans sa forme initiale. L'autre atout est que pour

notre projet, faire le moins d'opérations possibles est primordial pour avoir le moins d'erreurs d'arrondi possible sur des nombres à virgule.

4.2 Application aux fonctions trigonométriques

Comme les séries de Taylor pour sinus et cosinus ne contiennent que des puissances alternées, on pose $u = x^2$ on applique Horner sur u puis on remplace u par x^2.

Par exemple pour cosinus à l'ordre 8 et sinus à l'ordre 9, on obtient pour un x proche de 0:

$$cos(x) = \left(\left(\left(\frac{x^2}{8!} - \frac{1}{6!}\right)x^2 + \frac{1}{4!}\right)x^2 - \frac{1}{2}\right)x^2 + 1$$

$$\sin(x) = x\left(\left(\left(\left(\frac{1}{9!}x^2 - \frac{1}{7!}\right)x^2 + \frac{1}{5!}\right)x^2 - \frac{1}{3!}\right)x^2 + 1\right)$$

Puis on calcule tous les coefficients et on les écrit le plus précisément possible sur 32 bits.

4.3 Réduction de l'angle

Cette fois-ci, pour assurer que l'approximation est cohérente, il faut toujours ramener l'angle (le flotant f) le plus proche de 0 possible, les fonctions cosinus et sinus étant deux 2π-périodique, il suffit d'appliquer une soustraction de 2π à l'angle (ou une addition si l'angle est négatif) jusqu'à se trouver dans l'intervalle souhaité.

4.4 Tests

Table 2: Performances de la méthode Horner (sans réduction d'argument précise) pour le calcul du cosinus.

Intervalle	**Métrique**	**Horner**
$[-100, 100]$	Err. Abs. Max	2.00
	Err. ULP Moy.	3.38×10^7

Suite sur la page suivante...

Table 2: Performances de la méthode Horner... (suite)

Intervalle	**Métrique**	**Horner**
	Err. ULP Max	5.13×10^{11}
	Taux d'échec	98.894 %
$[-10\,000, 10\,000]$	Err. Abs. Max	2.00
	Err. ULP Moy.	2.88×10^{7}
	Err. ULP Max	4.88×10^{11}
	Taux d'échec	99.982 %
$[-100\,000, 100\,000]$	Err. Abs. Max	2.00
	Err. ULP Moy.	2.88×10^{7}
	Err. ULP Max	4.88×10^{11}
	Taux d'échec	99.982 %

Cette méthode n'est donc toujours pas satisfaisante, cette fois-ci l'erreur principale réside dans le stockage de la constante π, en effet dans notre algorithme nous la stockons sur 32 bits, cependant lorsque l'angle est grand, et donc qu'il faut appliquer utiliser un grand nombre de fois la constante π, de nombreuses erreurs d'arrondies se propagent, pour améliorer cela, on peut par exemple stocker cette constante avec une plus grande précision.

5 Implémentation finale de cos et sin

Toute cette partie et donc notre implémentation finale des fonctions trigonométriques a été fortement inspirée par le code source de la bibliothèque standard du langage C sur Linux (voir Bibliographie). On souhaite garantir une erreur de moins d'1 ULP dans toute cette partie pour des nombres "raisonnables".

5.1 Réduction d'argument

Cette étape est la plus importante puisqu'elle évite des tonnes de calculs.

On donne un angle x en radians, et on cherche exactement où on se trouve sur le cercle trigonométrique. L'objectif est d'écrire x de la forme suivante: $x = k \times \frac{\pi}{2} + r$, avec k qui représente le nombre de quarts de tours effectués (le quadrant) et r le petit reste, ie l'angle tq $|r| \leq \frac{\pi}{4}$.

5.1.1 Détermination de k

Pour s'assurer d'avoir $|r| \leq \frac{\pi}{4}$, on doit prendre k arrondi au plus proche entier. L'astuce est alors d'additionner 0.5 au flottant k puis de faire un cast vers un int. On est alors certain que notre r sera bien compris entre $[-\frac{\pi}{4}, \frac{\pi}{4}]$.

```
int get_k(float f){
    // valeur de 2/pi
    float PI2_INV = 0.63661977;

    float k_float = (f*PI2_INV) + 0.5;
    return (int)(k_float);
}
```

Algorithme de calcul de k

Remarque : On préfère écrire $k = x * \frac{2}{\pi}$ plutôt que $k = \frac{x}{\frac{\pi}{2}}$ puisque la multiplication est plus rapide que la division. D'ailleurs ce calcul vient du fait que l'on cherche combien de fois x fait de quarts de tours de cercle.

5.1.2 Algorithme de Cody-Waite étendu

Maintenant qu'on a obtenu k on cherche la valeur de r. Pour obtenir une valeur très précise on va stocker $\frac{\pi}{2}$ dans 3 variables différentes. On va définir $\frac{\pi}{2} = C_1 + C_2 + C_3$ avec C_1 (la partie haute) qui est la valeur de $\frac{\pi}{2}$ tronquée pour que ses derniers bits soient des zéros, C_2 (la partie intrmédiaire) aussi tronquée pour avoir des zéros à la fin, et finalement C_3 (partie basse) qui est le reste.

Le calcul du reste r se fait alors en trois étapes de soustraction :
1. La soustraction principale $r_1 = x - (k \times C_1)$. Comme C_1 a plein de zéros à la fin en binaire, la multiplication $k \times C_1$ donne un nombre "propre" qui ne déborde pas de la mémoire (pour la majorité des nombres). Ce calcul élimine la grosse partie de x (les tours de roue) sans trop d'erreur.
2. Maintenant on va faire une correction plus fine. On a enlevé les tours complets mais on a utilisé une valeur de tour (C_1) légèrement fausse à cause de la décomposition faite. On a oublié d'enlever C_2 à chaque tour, on l'enlève alors : $r_2 = r_1 - (k \times C_2)$.
3. La correction fine, l'ajustement final avec les dernières décimales de $\frac{\pi}{2}$: $r_{final} = r_2 - (k \times C_3)$.

Remarque : Les derniers bits de C_1 doivent être à 0 pour garantir que la multiplication $k \times C_1$ soit exacte, sans erreur d'arrondi. Un flottant (32 bits) a une précision limitée, sa mantisse peut contenir que 23 bits d'information. Lorsqu'on multiplie C_1 par k alors ça prendra plus de bits d'information que C_1. Si on choisissait C_1 pour être le plus proche possible de $\frac{\pi}{2}$, alors on pourait avoir un dépassement de capacité, donc le résultat sera arrondi. Il faut alors trouver un bon compromis pour que C_1 soit proche de $\frac{\pi}{2}$ mais aussi pour pouvoir multiplier par des grands nombres. La valeur choisie pour C_1 est alors 1.5703125 qui prend seulement 7 bits dans la mantisse, et donc laisse 16 bits pour k. Pour C_2 c'est pareil il fallait trouver un bon compromis, la valeur de la constante trouvée comporte 12 bits de zéros à la fin ce qui laisse assez pour des nombres assez grands.

5.1.3 Comparaison entre Cody-Waite à deux constantes et Cody-Waite étendu à trois constantes

Pour faire le choix du nombre de constantes utilisées pour décomposer $\frac{\pi}{2}$, on a testé avec un programme qui prend un million de flottants dans un intervalle choisi, qui calcule le sinus et cosinus pour chacun d'eux, et renvoie l'erreur absolue maximale obtenue parmi ces nombres comparée à la librairie Java, l'erreur ULP moyenne, l'erreur ULP max et le taux d'échec (ie les nombres dont le sinus/cosinus on plus d'un ULP de différence par rapport à la libraire Java).

On va donner des résultats du test seulement pour le sinus et pour 3 intervalles différents : $[-100, 100]$, $[-10\ 000, 10\ 000]$ et $[-100\ 000, 100\ 000]$. D'abord le test pour Cody-Waite avec 3 constantes et ensuite celui pour 2.

Table 3: Comparaison des performances de la réduction de Cody-Waite (3 vs 2 constantes) sur différents intervalles.

Intervalle	**Métrique**	**3 Constantes**	**2 Constantes**
$[-100, 100]$	Err. Abs. Max	1.19×10^{-7}	1.19×10^{-7}
	Err. ULP Moy.	0.20	0.24
	Err. ULP Max	125	22 878
	Taux d'échec	0.03 %	0.19 %

Suite sur la page suivante...

Table 3: Comparaison des performances... (suite)

Intervalle	**Métrique**	**3 Constantes**	**2 Constantes**
$[-10\,000, 10\,000]$	Err. Abs. Max	1.19×10^{-7}	1.79×10^{-7}
	Err. ULP Moy.	0.61	4.88
	Err. ULP Max	266 174	827 372
	Taux d'échec	1.19 %	16.54 %
$[-100\,000, 100\,000]$	Err. Abs. Max	1.19×10^{-7}	1.11×10^{-6}
	Err. ULP Moy.	2.19	29.92
	Err. ULP Max	217 314	573 483
	Taux d'échec	11.39 %	62.68 %

Comme on peut le voir, l'algorithme de Cody-Waite pour 2 constantes explose pour des grandes valeurs de x. Dans l'intervalle $[-100\ 000, 100\ 000]$, il y a plus de la moitié des calculs qui ont plus d'un ulp d'écart alors que seulement 10% pour celui à trois constantes. L'erreur moyenne à 2% pour la méthode à 3 constantes renforce la fiabilité.

Il faut aussi souligner que les cas qui posent beaucoup problème sont pour les valeurs de x qui sont multiples de π. Les valeurs gigantesques et aberrantes de l'erreur ULP max (200 000+) ne doivent pas alarmer. En effet, elles surviennent au voisinage des racines (donc des multiples de π où le sinus tend vers 0. A cet endroit, une erreur absolue minuscule (10^{-7}) représente une erreur relative énorme par rapport à un résultat quasi-nul (ex:10^{-30}), d'où un nombre énorme d'erreur ULP. C'est pour ça qu'il faut surtout regarder l'erreur absolue ici qui reste figée à $1,19 \times 10^{-7}$ (précision machine) pour la méthode à 3 constantes et donc qui ne perd aucune précision significative même à 100 000. Au contraire, la version à 2 constantes voit son erreur décupler sur le grand intervalle, ce qui nous indique la perte significative d'information.

Finalement, le coût d'une soustraction et multiplication supplémentaires est négligeable par rapport au gain de précision sur les grands nombres. C'est pourquoi nous avons décidé ce choix d'implémentation.

5.2 Cas des nombres très proches de 0

A cause du nombre limité de 32 bits pour écire un flottant, il existe un seuil tq en dessous de celui-ci on peut seulement approximer le sinus d'un nombre x par x.

Pour trouver ce seuil, il faut trouver la valeur tq l'ordinateur ne peut plus faire la différence entre la valeur réelle su sinus et le nombre x. Rappelons nous que la précision absolue de x s'écrit 2^{E-23}. Puisque $x \simeq 2^E$, on peut simplifier en disant que la précision est environ égale à $x \times 2^{-23}$. L'erreur de l'approximation est le premier terme négligé dans la série de Taylor, ie $\frac{x^3}{6}$. Pour que l'approximation soit valide, on veut que l'erreur soit plus petite que la précision. En remplaçant par les expressions et en simplifiant, on trouve : $x < \sqrt{6 \times 2^{-23}} \simeq 0.00084$. On a donc trouvé le seuil.

Remarque : Après tests, pour garantir une précision inférieurs à 1 ULP le seuil est de 0.0007.

Pour le cosinus maintenant on souhaiterait l'approximer par 1. Avec les mêmes calculs que pour le sinus on obtient un seuil d'environ 0.00034.

Remarque : Après tests, pour avoir une erreur de moins d'1 ULP on peut prendre un seuil de 0.00047.

5.3 Cas des très grands nombres

GLibc utilise une méthode très particulière pour gérer les très grands nombres. La librairie utilise la réduction de Payne-Hanek qui stocke des milliers de décimales de π dans un tableau pour faire des calculs plus précis.

Dans le cadre de notre projet, on est obligés d'accepter d'avoir des valeurs complètement fausses pour des très grands nombres. Les floats écrits sur 32 bits ne nous permettent pas une précision pour des nombres gigantesques. Alors pour gérer ce cas si, on aurait pu envoyer un message d'erreur disant que le nombre est trop grand (¿1 000 000 par exemple où dans l'intervalle $[-1\ 000\ 000, 1\ 000\ 000]$ nous avons une erreur absolue max de 3,12e-02). Mais on a décidé de ne rien faire et de renvoyer le résultat faux. L'utilisateur voulant tester des nombres très grands saura pertinemment que le résultat sera faux puisqu'on est sur des flottants écrits sur 32 bits.

6 Gestion des fonctions arctangente et arcsinus

Cette partie, comme celle pour les fonctions sinus et cosinus a été inspirée par glibc. L'objectif est toujours d'avoir au plus un ULP.

6.1 Implémentation de la fonction arctangente

Comme pour les fonctions sinus et cosinus, la fonction arctangente est approchée à l'aide d'un polynôme de Taylor, évalué efficacement grâce à la méthode de Horner.

Ainsi, on obtient après application de la méthode de Horner l'approximation suivante:

$$\arctan(x) \approx x\left(1+x^2\left(-\frac{1}{3}+x^2\left(\frac{1}{5}+x^2\left(-\frac{1}{7}+x^2\left(\frac{1}{9}+x^2\left(-\frac{1}{11}+x^2\left(\frac{1}{13}-\frac{x^2}{15}\right)\right)\right)\right)\right)\right)\right)$$

Ensuite, le flottant en entrée est toujours ramené positif et dans un petit intervalle pour que l'approximation polynomiale reste correcte.

- Pour les très petites valeurs, on utilise l'approximation $\arctan(x) \simeq x$. J'ai pris 0.0007 comme borne supérieure comme pour sinus, valeur décidée après tests.

- Pour les grandes valeurs ($x > tan(\frac{3\pi}{8})$), on applique :

$$\arctan(x) = \frac{\pi}{2} - \arctan\left(\frac{1}{x}\right)$$

afin de "ramener" la valeur évaluée proche de 0 pour pouvoir utiliser l'approximation via Taylor de manière correcte.

- Pour les valeurs intermédiaires, autour de 1 (entre $\tan(\frac{\pi}{8})$ et $\tan(\frac{3\pi}{8})$)pour lesquelles utiliser la formule précédente n'aurait pas de sens puisque la valeur évaluée ne serait pas proche de 0, on utilise la transformation :

$$\arctan(x) = \frac{\pi}{4} + \arctan\left(\frac{x-1}{x+1}\right)$$

Remarque : Les valeurs aux bornes des différents intervalles ont été choisies d'après les codes de la librairie glibc. Elles garantissent que le polynôme de Taylor approxime pour des valeurs inférieures à 0.414 en valeur absolue, donc l'ordinateur peut calculer directement $\arctan(x)$ sans perdre de précision. La zone intermédiaire (entre les deux bornes) est la zone centrée autour de 45 (où $\tan(45) = 1$). Les valeurs de x sont comprises entre 0.414 et 2.414, c'est trop grand pour une bonne précision.

6.2 Implémentation de la fonction arcsinus

La fonction arcsinus est implémentée à partir de la fonction arctangente grâce à la relation trigonométrique suivante, valable pour $x \in]-1, 1[$:

$$\arcsin(x) = \arctan\left(\frac{x}{\sqrt{1-x^2}}\right)$$

De plus, la parité de la fonction est exploitée : pour une valeur négative, on utilise la relation :

$$\arcsin(-x) = -\arcsin(x)$$

afin de ne traiter que les cas positifs et simplifier les calculs.

La fonction arcsin étant définie sur $[-1, 1]$, on ne peut pas utiliser la formule avec l'arctangente écrites quelques lignes au dessus. On teste alors avant si $x = 1$ (resp. $x = -1$) on renvoie $\frac{\pi}{2}$ (resp. $-\frac{\pi}{2}$).

Ensuite, pour des nombres en entrée en dehors de l'intervalle de définition, on devrait renvoyer un Nan. En Deca, on ne l'a pas alors on renvoie seulement un message d'erreur puis 0 pour pas faire planter le programme.

Dans la formule principale utilisant l'arctangente, on utilise l'opération "racine carré" qu'on doit donc implémenter, pour cela on utilise l'algorithme de Newton-Raphson.

6.3 Implémentation de la fonction racine carrée

6.3.1 Algorithme de Newton-Raphson

La méthode de Newton-Raphson est un algorithme permettant de trouver des solutions approchées à des équations du type $f(x) = 0$. L'idée est de remplacer la courbe représentative de la fonction par sa tangente.

Intuition géométrique : On cherche l'endroit où une courbe coupe l'axe des abscisses. Pour cela, on va faire comme suit :
- On choisit un point de départ arbitraire x_0. - On trace la tangente à la courbe en ce point.
- On regarde le point x_1 qui est l'intersection entre la tangente et l'axe es abscisses.
- On répète le processus à partir de x_1 et ainsi de suite.

Il faut faire attention à certains exemples de fonctions dont cette formule ne fonctionne pas, ex : fonction dont la dérivée s'annule, fonction qui a une pente positive, puis négative...

On peut alors énoncer le théorème suivant : Soit $f \in C^2(\mathbb{R})$ qui passe au moins une fois par 0, tq f' et f'' gardent le même signe sans s'annuler. Alors l'équation $f(x) = 0$ possède une unique racine c. Posons x_0 dans $\mathbb{R}$ et définissons (x_n) par récurrence en posant :

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

Alors la suite (x_n) converge vers c.

6.3.2 Application au calcul de la fonction racine carrée

Le calcul de la racine carrée est réalisé à l'aide de la méthode itérative de Newton-Raphson présentée précédemment, qui permet de résoudre efficacement l'équation :

$$y^2 = x$$

Cette équation est équivalente à chercher un zéro de la fonction :

$$f(y) = y^2 - x$$

La méthode de Newton conduit alors à la relation de récurrence suivante :

$$y_{n+1} = y_n - \frac{f(y_n)}{f'(y_n)} = \frac{1}{2}\left(y_n + \frac{x}{y_n}\right)$$

Cette formule est utilisée directement dans notre implémentation.

Initialisation et convergence L'algorithme est initialisé avec la valeur $y_0 = 1$. Ce choix simple garantit une convergence rapide pour l'ensemble des valeurs positives usuelles.

La méthode de Newton présente une convergence quadratique : à chaque itération, le nombre de chiffres corrects est approximativement doublé. Après tests, on prend 15 itérations pour une valeur précise et rapide.

Gestion des cas particuliers Plusieurs cas particuliers sont traités explicitement :

- Si $x < 0$, la racine carrée n'est pas définie dans $\mathbb{R}$. La fonction retourne alors une valeur NaN en provoquant une opération arithmétique invalide. En Deca, ça va stopper le programme en renvoyant une erreur de stack overflow. On a laissé comme ça car cette fonction ne sera jamais appelée pour des nombres négatifs puisqu'elle est seulement appelée dans la fonction asin qui traîte déjà ce cas là.
- Si $x = 0$, la fonction retourne directement 0 afin d'éviter toute division inutile.

6.4 Tests des fonctions arctan et arcsin

Nous allons passé le même test que pour sinus, cosinus et pour l'ULP (voir section 7) pour tester nos algorithmes de arcsin et arctan, donc en Java.

Pour arctan, comme pour cosinus, on va tester sur 3 intervalles $[-1, 1]$, $[-100, 100]$ et $[100\ 000, 100\ 000]$.

Table 4: Performances de la fonction Arctan sur différents intervalles.

Intervalle	**Métrique**	**Arctan**
$[-1, 1]$	Err. Abs. Max	8.94×10^{-8}
	Err. ULP Moy.	0.3599
	Err. ULP Max	3.00
	Taux d'échec	1.09 %
$[-100, 100]$	Err. Abs. Max	1.19×10^{-7}
	Err. ULP Moy.	0.3653
	Err. ULP Max	2.00
	Taux d'échec	0.01 %
$[-100\,000, 100\,000]$	Err. Abs. Max	1.19×10^{-7}
	Err. ULP Moy.	0.3670
	Err. ULP Max	2.00
	Taux d'échec	0.00 %

Ces résultats nous indiquent que la fonction arctangente est parfaitement précise. Il n'y a pas grand chose à dire de plus sur ces résultats.

Remarque : Les intervalles pour arctan ont été choisi de cette façon puisque c'est autour de 0 qu'il y a le plus de chance que ce ne soit pas précis, à cause de la forte pente de cette fonction autour de cette valeur et aussi comme l'ULP est minuscule vers cet endroit.

Pour arcsinus, comme la fonction est définie seulement sur $[-1, 1]$, on va seulement tester sur cette intervalle. Hors cet intervalle on aurait eu beaucoup de valeurs inutiles qui auraient faussées le test en renvoyant 0.0.

Table 5: Performances de la fonction Arcsin sur l'intervalle $[-1, 1]$.

Intervalle	**Métrique**	**Arcsin**
$[-1, 1]$	Err. Abs. Max	8.34×10^{-7}
	Err. ULP Moy.	0.4540

Suite sur la page suivante...

Table 5: Performances de la fonction Arcsin... (suite)

Intervalle	**Métrique**	**Arcsin**
	Err. ULP Max	7.00
	Taux d'échec	2.95 %

Pour arcsinus, on remarque une moins bonne imprécision, très légère mais qu'il faut souligner quand même. La valeur ici la moins imprécise se situe autour de -1, à -0,999827. En fait, la nombre passé en paramètre à l'intérieur de la racine carrée dans la relation trigonométrique reliant arcsin et arctan est très très proche de 0. Le nombre $\frac{x}{\sqrt{1-x^2}}$ diverge alors beaucoup et donc le résultat peut perdre en précision. La fonction implémentée reste tout de même très fiable, puisque l'erreur à cet endroit est de 8.34×10^{-7}.

7 Algorithme de calcul de l'ulp

7.1 Explication de l'algorithme

Le but de l'algorithme est de trouver la puissance de 2 la plus proche du nombre par des multiplications/ divisions successives.

Pour tout entier naturel i, deux flottants successifs dans $[2^i, 2^{i+1}[$ sont distants de la même valeur par définition de la norme IEEE 754.

Par exemple, entre 1 et 2 (2 exclu), le plus petit nombre est 1 qui s'écrit en binaire 0011 1111 1000 0000 0000 0000 0000 0000. Le suivant s'écrit 0011 1111 1000 0000 0000 0000 0000 0001 et qui vaut $1 + 2^{-23}$. La différence entre les deux est 2^{-23}. De même pour le dernier nombre de l'intervalle.

La formule de l'ULP pour un nombre $x = m \times 2^E$ est $ulp(x) = 2^{E-23}$ puisque entre 2 nombres successifs c'est le dernier bit de la mantisse qui va changer, d'exposant 2^{-23}, auquel on mutltiplie l'exposant réel du nombre pour avoir la bonne valeur de l'exposant final. Cette formule peut aussi s'écrire $ulp(x) = 2^E \times 2^{-23} = \epsilon \times 2^E$, avec ϵ qui correspond au premir intervalle $[1, 2[$.

Le problème est qu'en Deca on ne connaît pas E. On a juste le nombre x, mais on ne peut pas avoir directement son exposant. Il faut alors trouver un moyen de déterminer le facteur 2^E.

Regardons cer qu'il se passe quand on multiplie x par 2 :
- Si x devient $2x$, alors ça revient à ajouter 1 à l'exposant E (car si $x = m \times 2^E$, $2x = m \times 2^{E+1}$).
- L'ULP devient donc $\epsilon \times 2^{E+1}$, ie le double de l'ULP précédent.

L'idée de l'algorithme est alors de se ramener à l'intervalle $[1, 2[$ puisque nous connaissons déjà l'ULP dans celui-ci, tout en divisant/multipliant par 2 l'ULP réel du nombre à chaque fois qu'on mutiplie/divise le nombre.

Par exemple, prenons $x = 12.5$. Déroulé de l'algorithme:
1. 12.5 n'est pas dans $[1, 2[$, on le divise par 2, $x \to 6.25$ et on multiplie son ULP par 2.
2. 6.25 n'est toujours pas dans $[1, 2[$, on le divise par 2 et on multiplie son ULP par 2.
3. 3.125 n'est toujours pas dans $[1, 2[$, on le divise par 2 et on multiplie son ULP par 2.
4. 1.5625 est bien dans $[1, 2[$, on sait que son ULP vaut ϵ et on s'arrête.
5. On a divisé 3 fois par 2 l'ULP de x, donc finalement $ULP(x) = \epsilon \times 2^3$.

7.2 Ecriture de l'algorithme en Deca et tests

```
float ulp(float x) {

    float e = 1.1920929E-7; //valeur de epsilon

    x = abs(x);

    if (x < 1.17549435E-38) {
        return 1.4E-45;
    }

    while (x >= 2.0) {
        x = x / 2.0;
        e = e * 2.0;
    }

    while (x < 1.0) {
        x = x * 2.0;
        e = e / 2.0;
    }
```

```
20
21      return e;
22  }
```

Méthode ULP en Deca

L'écriture de l'algorithme est plutôt simple, et fonctionne à la perfection. Le test précédent sur le cosinus dans la partie 5.1.3 exécuté sur l'intervalle $[-100\,000, 100\,000]$ pour tester l'ulp nous renvoie une erreur relative max de 1.19×10^{-7} et un taux d'échec de 0,00 pourcent. En fait, pour aucun flottant en entrée il y a plus d'un ulp de différence entre notre algorithme de l'ulp et l'ulp de la librairie Math de Java, donc l'algo est fiable à 100 pourcents.

8 Tests de la librairie en Deca et conclusion

8.1 Tests de l'implémentation en Deca de la librairie avec notre compilateur

Dans cette section, on va voir que les résultats vus en Java sont quasi identiques en Deca. La seule différence est l'affichage des nombres flottants, en Deca ils sont toujours écrits en notation scientifique et ont toujours 6 chiffres (exemple : -9.99361e-01). En Java, ça dépend mais lorqu'il n'y a pas trop de 0 après la virgule ils sont écrits sous forme "standard" et ont 7 chiffres significatifs au plus, ie le max pour des floats (exemple : -0.9993608 pour le même nombre qu'en Deca).

Voici quelques résultats de test en Deca et le même en Java en utilisant la librairie Math de Java :

Table 6: Comparaison détaillée des résultats : Lib Math Deca vs Java

Fonction	Entrée (Test)	Résultat Deca	Résultat Java
Sinus	$-50\,000$	9.99840×10^{-1}	0.9998402
	$\pi/2$	$1.00000 \times 10^{+0}$	1.0

Suite sur la page suivante...

Table 6: Comparaison détaillée des résultats... (suite)

Fonction	Entrée (Test)	Résultat Deca	Résultat Java
	-0.03654	-3.65319×10^{-2}	-0.03653187
	$300\pi/2$	2.40805×10^{-5}	2.408067×10^{-5}
Cosinus	$100\,000$	-9.99361×10^{-1}	-0.9993608
	-0.69842	7.65859×10^{-1}	0.7658591
	$101\pi/2$	-7.39502×10^{-6}	-7.395083×10^{-6}
	-163.2154	9.89153×10^{-1}	0.9891526
ArcTan	0.005	4.99996×10^{-3}	0.004999958
	$15\,890.25$	1.57073×10^{0}	1.5707334
	-548.2	-1.56897×10^{0}	-1.5689722
	0.0	$0.00000 \times 10^{+0}$	0.0
ArcSin	0.2548	2.57641×10^{-1}	0.25764087
	-26 (Hors D.)	$0.00000 \times 10^{+0}$	`NaN`
	0.0009852	9.85200×10^{-4}	9.852002×10^{-4}
	-0.57	-6.06506×10^{-1}	-0.6065059
ULP	12.424	9.53674×10^{-7}	9.536743×10^{-7}
	$-5\,888\,992$	5.00000×10^{-1}	0.5
	-0.00069	5.82077×10^{-11}	5.820766×10^{-11}
	$2\,641.549$	2.44141×10^{-4}	2.441406×10^{-4}

Le tableau testant seulement quelques valeurs nous permet quand même de nous indiquer que tous les tests réalisés en Java fonctionnent et sont bien représentatifs.

Un point que nous avons changé par rapport à notre implémentation en Java est la gestion des cas hors intervalle de définition pour la fonction arcsinus. Nous n'avons pas les NaN (Not a Number) dans notre compilateur, ainsi l'astuce de renvoyer $\frac{0}{x-x}$ ne fonctionnait pas et stoppait le programme dès qu'on testait pour ce cas. Ainsi, nous avons décidé de print un message

d'erreur ("Erreur de domaine : votre float doit être dans [-1, 1] ") puis de renvoyer 0.0. La gestion des nombres négatifs cependant n'a pas été modifiée pour la fonction sqrt puisque ce n'est pas une fontion terminale dans notre librairie (comme demandé dans le cahier des charges de l'extension TRIGO), et ainsi ce cas n'arrive jamais puisque la fonction sqrt est uniquement appelée dans la fonction arcsinus qui gère les nombres négatifs en utilisant l'imparité de la fonction.

Remarque : Comme cela n'a pas été demandé, nous n'avons pas implémenté la fonction tangente, ni la fonction arccosinus. Cependant leur implémentation aurait été directe, surtoût pour tangente par sa définition : $tan(x) = \frac{sin(x)}{cos(x)}$ pour x différent de $\pi/2 + k\pi$ avec k dans $\mathbb{Z}$. On a testé cette fonction en Java et les résultats étaient impeccables, comme les fonctions sinus et cosinus. Il fallait seulement enlever les cas où le cosinus était très proche de 0 parce que ça divergeait beaucoup trop et donc l'erreur ne voulait plus rien dire si on comparait avec la tangente de la librairie Math de Java.

8.2 Conclusion sur la librairie

Notre librairie TRIGO est fonctionnelle pour les cinq fonctions demandées. Je pense même que l'on ne pouvait pas faire mieux au niveau de la précision (voir parties précédentes). Un point qui n'a pas été abordée cependant est le temps de compilation et d'exécution de chacune d'elles, mais on a décelé aucun problème là dessus même pour le gros test (en Java) sur un million de valeurs expliqué plus tôt dans la documentation, les temps étaient très courts.

9 Bibliographie

– How do C and Python compute sin and cos?, par Matthew Tower.
– What Every Computer Scientist Should Know About Floating-Point Arithmetic, par David Goldberg.
– Nos anciens cours de prépa sur les séries de Taylor et sur toutes les formules trigonométriques
– Slides sur Les calculs sur les flottants, fournies par l'ENSIMAG.
– Code source du sinus de glibc, bibliothèque standard de C sur Linux, par IBM.

– Algorithme de Horner, Bibmath
– Méthode Newton-Raphson, et approximation de racine de 2 par la méthode d'Héron d'Alexandrie, Bibmath